

TEMA 1 – Ciclo de Vida

Objetivo del tema

En este tema aprenderás uno de los aspectos más importantes de las App Android: su ciclo de vida

1.1 El Ciclo de Vida de una Actividad

Cuando creas un programa Java en Eclipse, ese programa tiene un control absoluto sobre su propio ciclo de ejecución. Para entendernos, queremos decir que ese programa se arranca, se para y se pone en pausa cuando él mismo quiere hacerlo, sin más.

Bien. Eso no pasa en Android.

Las Apps en Android tienen **muy poco control** sobre su ciclo de ejecución. Deben estar atentas a sus cambios de estado y a reaccionar en consecuencia. A demás, las Apps están preparadas para la finalización repentina de su ejecución. Piensa que tu App estará funcionando en un **dispositivo móvil**, por lo tanto, el usuario puede encender otra App en cualquier momento, ponerla en segundo plano, volver a ella, recibir una llamada, etc.

Por defecto, cada aplicación Android se ejecuta **en su propio proceso** (ver Procesos en PSP), cada uno con su propia instancia en la máquina virtual de Android (Dalvik o ART). La forma que tiene Android de **seleccionar** el proceso más adecuado para ponerse en ejecución es muy especial. Android está diseñado para dar prioridad a las interacciones del usuario excluyendo todo lo demás. Por tanto, a veces hay Apps y procesos en segundo plano que dejan de ejecutarse por completo, incluso sin previo aviso, para que se puedan liberar recursos para las Apps con las que el usuario está trabajando y prestando el 100% de su atención.

El orden en el que los procesos de las Apps **son detenidos por el sistema operativo** viene determinado por su prioridad, que se establece en base al estado en el que se encuentre la activity de mayor prioridad de dicha App.

Prioridad	Proceso	
Crítica	Activo	Aquellos con los que el usuario está interactuando en este momento. Android liberará recursos para asegurarse de que responden sin latencia. Sólo se los detiene si no hay más remedio.
Alta	Visible	Visible pero inactivo, ya sea porque la interfaz está oculta o no se está interaccionando con ellos. Puede que haya otra interfaz por encima, haya aparecido un diálogo o no ocupa toda la pantalla.
	De Servicio	Los servicios permiten que haya una ejecución de código sin interfaz con el usuario.
Baja	Inactivos	Actividades que ni son visibles ni están realizando tareas, y que tampoco ejecutan ningún servicio. El orden en el que se detendrán depende del tiempo que lleven inactivos.
	Vacíos	Un proceso terminado pero cacheado en memoria por Android, no vaya a ser que el usuario quiera volver a lanzarlo. Por supuesto, son de prioridad mínima.

Aquellos procesos de menor prioridad serán los primeros en ser ‘purgados’ para liberar recursos. Y se entregará el procesador a aquellos procesos con mayor prioridad. Por lo demás, esto funciona muy similar a lo mencionado en la asignatura de PSP.

Por otro lado, están las actividades dentro de cada Proceso. Una **activity** individual es (normalmente) cada una de las **pantallas** que forman nuestra App. Cada una de ellas tiene un **layout** relacionado (normalmente). Cuando arrancamos una App se ejecuta la actividad configurada como principal en el **AndroidManifest.xml**, de haber varias, y podremos ir de una actividad a otra según nos convenga.

En Android **no existe** el método main como en Java. Las activity tienen un ciclo de ejecución que hay que conocer. Aunque para la mayoría de las Apps sencillas nos basta con poner código en el método **onCreate ()**, hay que entender cómo funciona todo esto.

Toda actividad dentro de una App tiene un **estado**. Este estado servirá para determinar su prioridad en el contexto de su App padre. Recuerda que la prioridad de la App depende de la prioridad de su actividad de mayor prioridad. El estado viene determinado por su posición en la **pila de actividades**. Esta es una pila LIFO (Last-In-First-Out) en la que se encuentran todas las activity actualmente en ejecución. Cuando empiezas una nueva activity, la que se estaba mostrando en ese momento pasa al tope de la pila. Si le damos a 'volver' o cerramos esa activity, la activity del tope de la pila saldrá de ella y pasará a ser la activa.

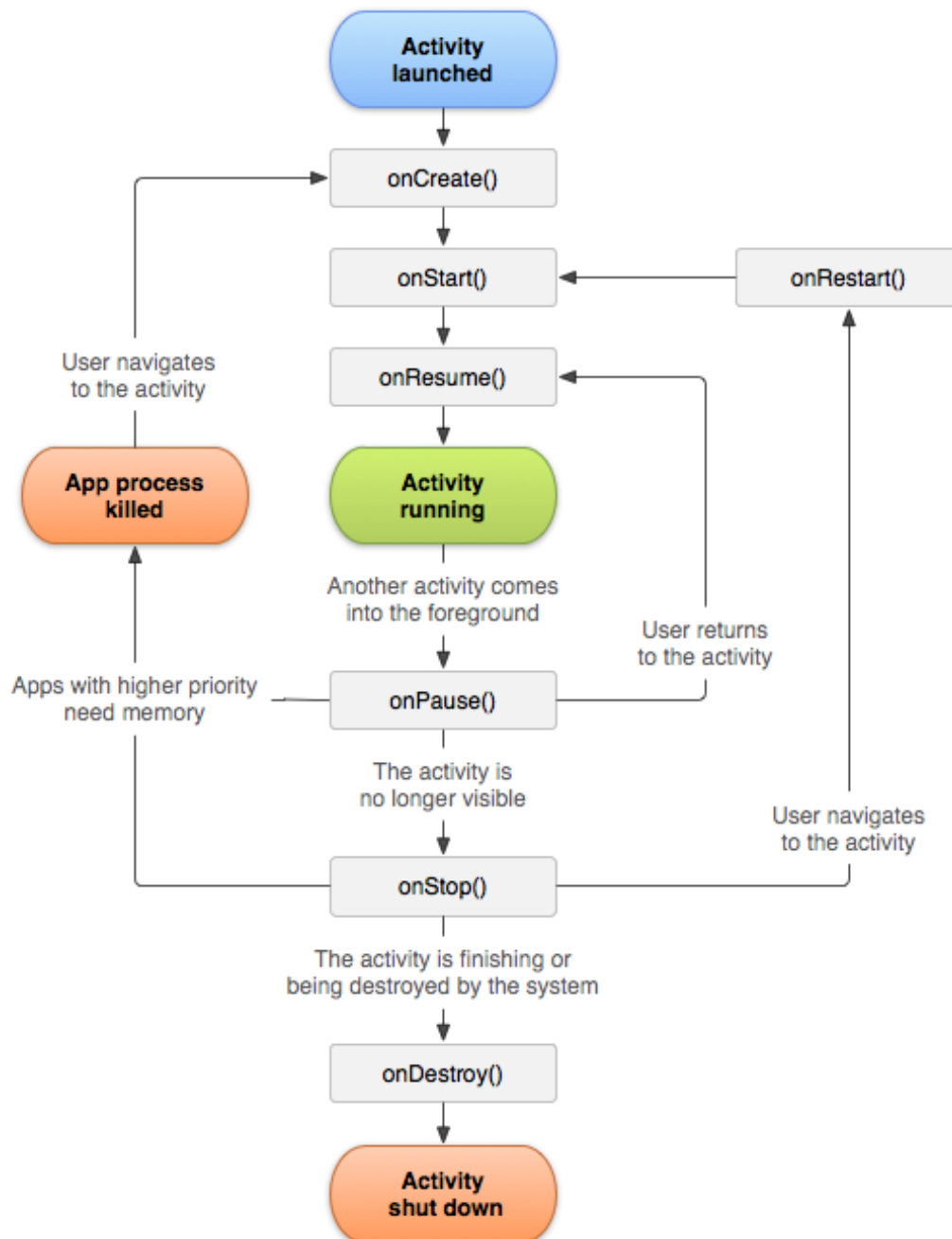
Estados	
Activa	La activity se está ejecutando en este momento, es visible, tiene el foco y el usuario puede interaccionar con ella. Android intentará por todos los medios mantener esta activity en ejecución.
En pausa	La activity está activa, pero sin foco. Suele ocurrir que su interfaz está por debajo de otra, o que no esté en pantalla completa. No puede recibir interacciones del usuario.
Detenida	La activity no es visible, pero permanece en memoria con todos sus datos. La pega es que puede ser eliminada de memoria en cualquier momento. Normalmente, cuando una activity pasa a detenida, se suelen almacenar los datos y el estado de la interfaz.
Inactiva	La activity pasa a inactiva cuando se ha terminado su ejecución o todavía no se ha iniciado. Se han sacado de la pila y deben de ser lanzadas de nuevo.

Todo esto es transparente para el usuario, que se limita a mover el dedo sobre la pantalla del móvil sin más. Pero los **programadores** podemos gestionar estos cambios de estado mediante eventos. Para ello, simplemente sobrecargamos el método correspondiente de la clase Activity.

Dicho de otra forma. Si conocemos el ciclo de vida de una Actividad, sus estados, y sus cambios, podremos aprovecharnos de ellos para añadir comportamientos a nuestra App.

Esta es la forma que tienen los juegos de móvil de pausarse automáticamente y guardar tus progresos cuando contestas a una llamada.

Fíjate en el esquema de la página siguiente:



Este gráfico muestra un esquema de los **estados**, los **eventos** y la **secuencia** en la que se producen. Por ejemplo, al arrancarse una actividad, se lanza un evento de forma automática que ejecuta el método **onCreate ()** de tu App. Es por eso por lo que nosotros añadimos código en este método para nuestros ejercicios. Pero la cosa no se detiene ahí: a continuación, se ejecuta el método **onStart ()**, luego el **onResume ()**, y por fin, la Actividad pasa a estar en ejecución.

Más tarde, el usuario lanza otra App. Esto provoca que el estado de nuestra App cambie y se lance el método **onPause ()**. Y así sucesivamente.

Estos métodos no aparecen en la clase Kotlin de la Actividad a excepción del `onCreate ()`, pero eso no significa que 'no existan'. Se encuentran en la superclase, en espera de que los redefinas.

A modo de resumen:

- **onCreate ()** – Se añade aquí el código necesario para inicializar el interfaz.
- **onStart ()** – Se llama cuando la actividad pasa a estado visible.
- **onResume ()** – Se llama cuando la actividad pasa a estado activo. Se suelen poner aquí los hilos (Threads) que mueven muñequitos de los videojuegos (por ejemplo).
- **onPause ()** – Se llama al pausar una actividad. Todo lo que haga onResume () tiene que deshacerse en onPause ().
- **onStop ()** – Se llama cuando una actividad deja de ser visible. Todo lo que se haga en onStart () debería deshacerse en onStop ().
- **onRestart ()** – Se llama cuando una actividad vuelve a ser visible después de haber estado no visible.
- **onDestroy ()** – Se llama cuando se va a destruir la actividad. Debería deshacer todo lo hecho en el onCreate ().

Ejercicio 15

Crea una App sencilla. No hace falta que le añadas ningún widget. A continuación, añade los métodos mencionados en este documento referentes al ciclo de vida de una Actividad. Añade en cada método una traza de Logcat con un mensaje para identificarla. A continuación, emula la App e intenta disparar los métodos colocando la App en segundo plano, volviendo a ella, etc.